

機器學習算法分析實例

A Tutorial on Analyzing Machine Learning Algorithms

吳沛遠

臺灣大學電機系

peiyuanwu@ntu.edu.tw



■ 研究興趣

機器學習(核方法、深度學習、隱私維護等)、統計學習理論、影像處理與電腦視覺。

■ 相關產官學合作

- 國科會 – 隱私維護機器學習、強健性分析
- 國家太空中心 – 福衛影像除霧
- 中華電信數據分公司 - 自然場景中英文字辨識
- 臺灣積體電路公司 – 自動晶圓檢測與良率提升
- 工研院 - 多方安全計算應用於深度學習
- 華碩 - 機器學習優化伺服器風扇控制邏輯
- 佳世達 – 口腔掃描儀結構光點雲重建

■ 簡歷

- B.S. National Taiwan University, 2009
- M.S. Princeton University, 2012
- Ph.D. Princeton University, 2015
- Principle engineer, TSMC, 2015-2017
- Associate professor, National Taiwan University, 2017-now

Outline

- LLM reasoning
- Gradient Descent with Adversarial Corruption
- Linear bandit upper confidence bound
- Reinforcement learning

LLM Reasoning

LLM Reasoning – Chain-of-thought

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

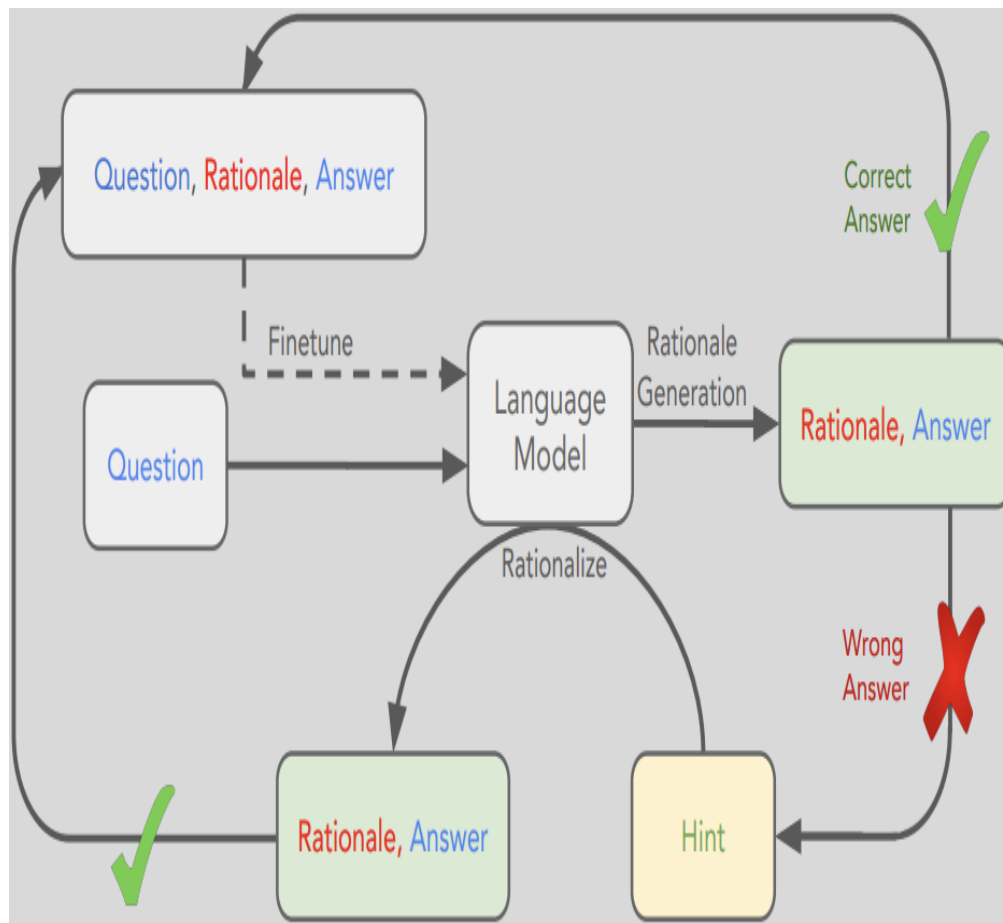
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Self-Taught Reasoner

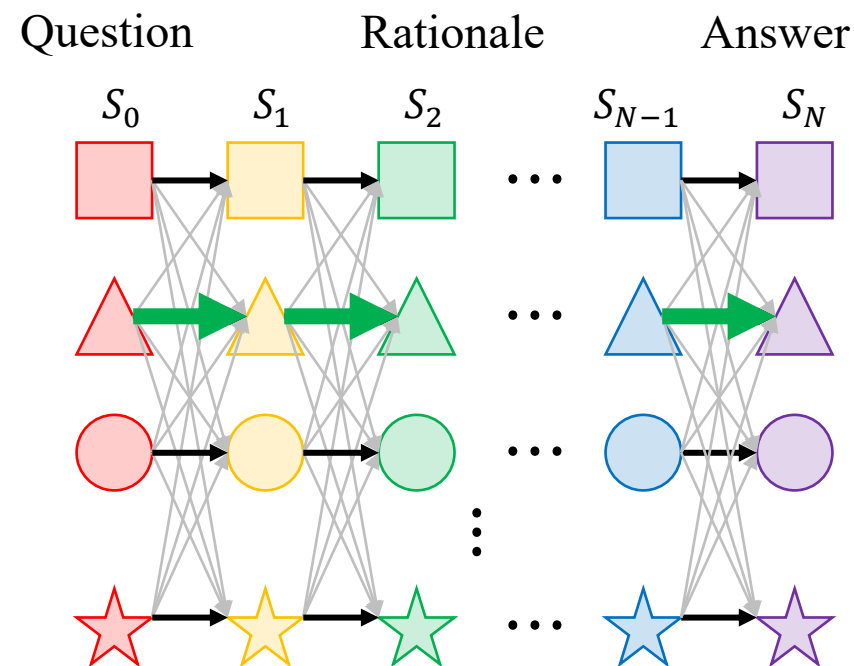
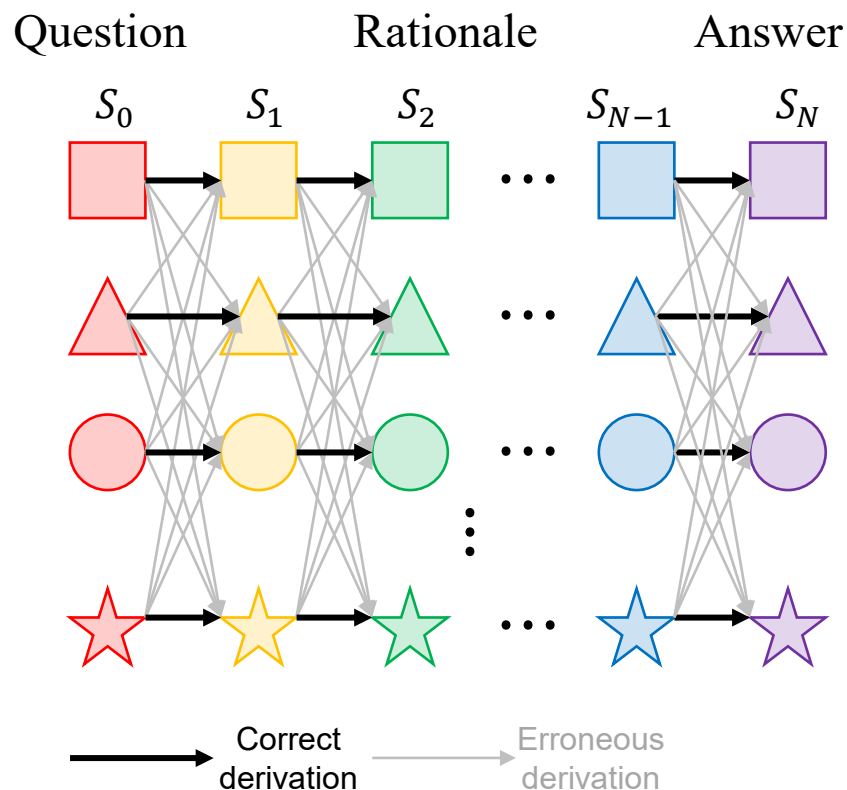


- Such generation process can be automated provided the final answer is known (No need to annotate the correctness of rationale) 👍
- May include wrong rationale (but happens to have the correct final answer) to the training data 🚫

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. **STaR: self-taught reasoner bootstrapping reasoning with reasoning**. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1126, 15476–15488.

Self Taught Process – A simplified model

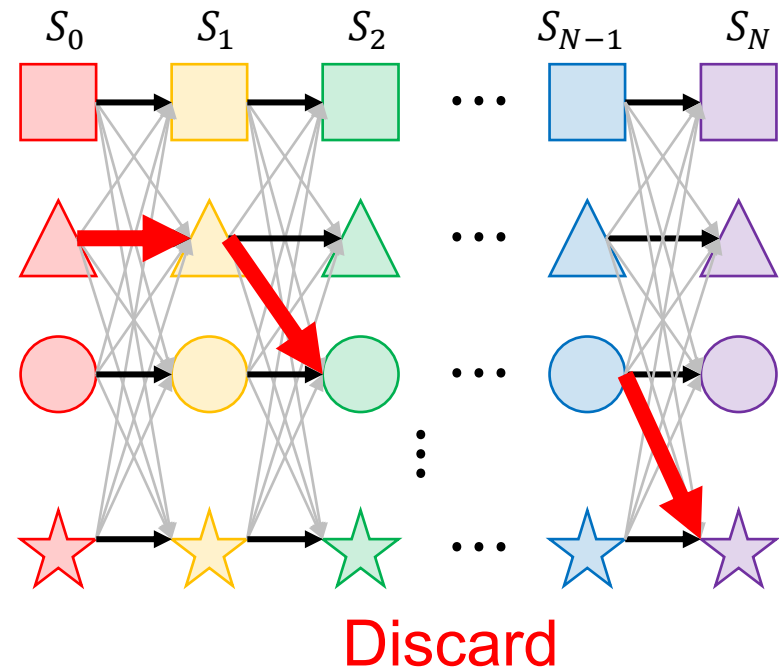
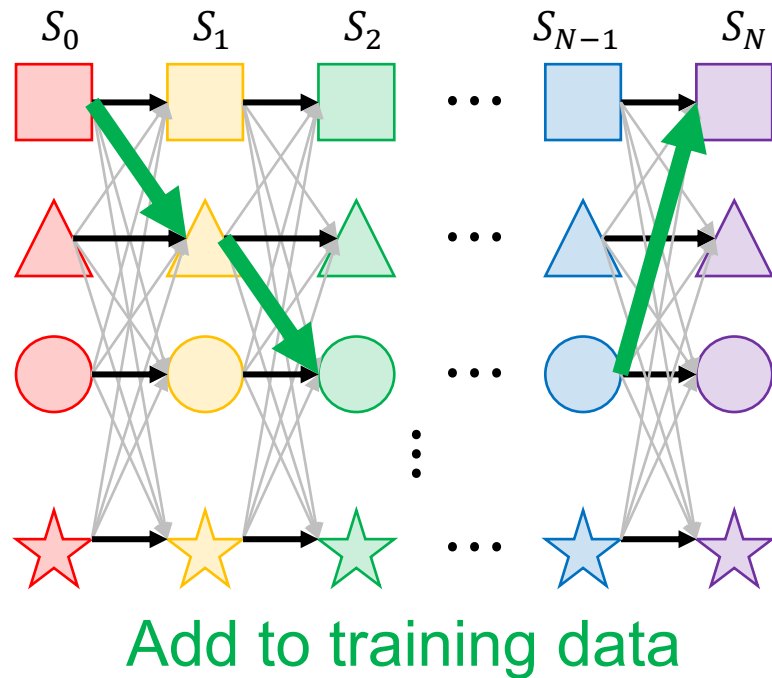
We consider a simplified model, where the chain-of-thought contains N steps of derivations



Perfect rationale: Each step follows the correct derivation.

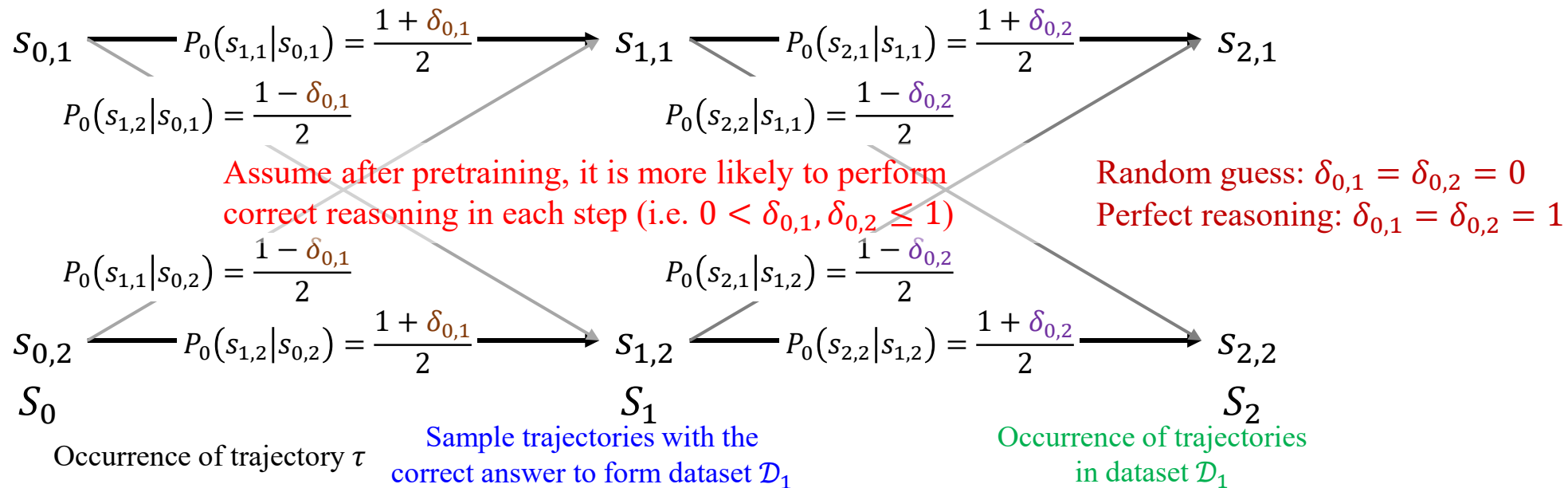
Self Taught Process – Data Collection

Suppose we generate rationales and add those with the correct final answer (no matter the derivation) to training data



Does such process improve rationale capability?

Toy Example



$$p(\tau) = \begin{cases} \frac{1}{2} \left(\frac{1+\delta_{0,1}}{2} \right) \left(\frac{1+\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,1}, s_{1,1}, s_{2,1}) \\ \frac{1}{2} \left(\frac{1+\delta_{0,1}}{2} \right) \left(\frac{1-\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,1}, s_{1,1}, s_{2,2}) \\ \frac{1}{2} \left(\frac{1-\delta_{0,1}}{2} \right) \left(\frac{1-\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,1}, s_{1,2}, s_{2,1}) \\ \frac{1}{2} \left(\frac{1-\delta_{0,1}}{2} \right) \left(\frac{1+\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,1}, s_{1,2}, s_{2,2}) \\ \frac{1}{2} \left(\frac{1-\delta_{0,1}}{2} \right) \left(\frac{1+\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,2}, s_{1,1}, s_{2,1}) \\ \frac{1}{2} \left(\frac{1-\delta_{0,1}}{2} \right) \left(\frac{1-\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,2}, s_{1,1}, s_{2,2}) \\ \frac{1}{2} \left(\frac{1+\delta_{0,1}}{2} \right) \left(\frac{1-\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,2}, s_{1,2}, s_{2,1}) \\ \frac{1}{2} \left(\frac{1+\delta_{0,1}}{2} \right) \left(\frac{1+\delta_{0,2}}{2} \right) & \text{if } \tau = (s_{0,2}, s_{1,2}, s_{2,2}) \end{cases}$$

$$p(\tau | \tau \in \mathcal{D}_1) = \begin{cases} \frac{(1+\delta_{0,1})(1+\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,1}, s_{1,1}, s_{2,1}) \\ \frac{(1-\delta_{0,1})(1-\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,1}, s_{1,2}, s_{2,1}) \\ \frac{(1-\delta_{0,1})(1-\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,2}, s_{1,1}, s_{2,2}) \\ \frac{(1+\delta_{0,1})(1+\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,2}, s_{1,2}, s_{2,2}) \end{cases}$$

Toy Example

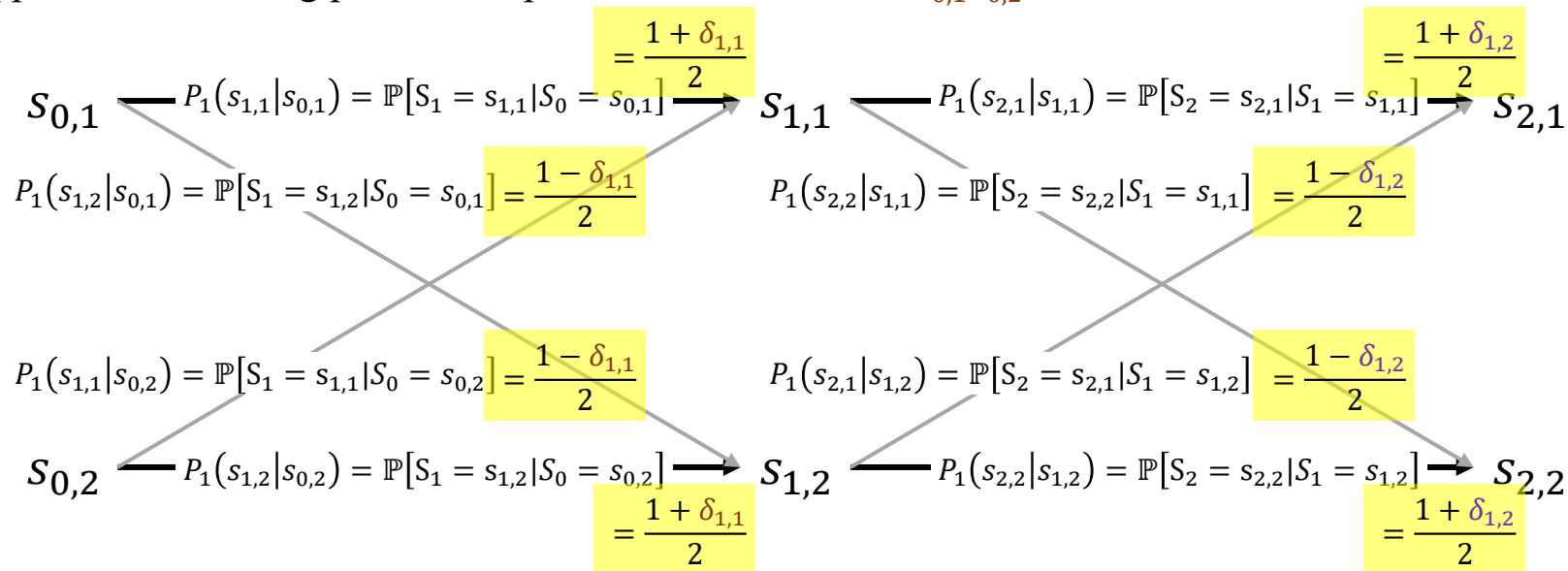
Occurrence of trajectories in dataset $\mathcal{D}_1$:

$$p(\tau | \tau \in \mathcal{D}_1) = \begin{cases} \frac{(1+\delta_{0,1})(1+\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,1}, s_{1,1}, s_{2,1}) \\ \frac{(1-\delta_{0,1})(1-\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,1}, s_{1,2}, s_{2,1}) \\ \frac{(1-\delta_{0,1})(1-\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,2}, s_{1,1}, s_{2,2}) \\ \frac{(1+\delta_{0,1})(1+\delta_{0,2})}{4(1+\delta_{0,1}\delta_{0,2})} & \text{if } \tau = (s_{0,2}, s_{1,2}, s_{2,2}) \end{cases}$$

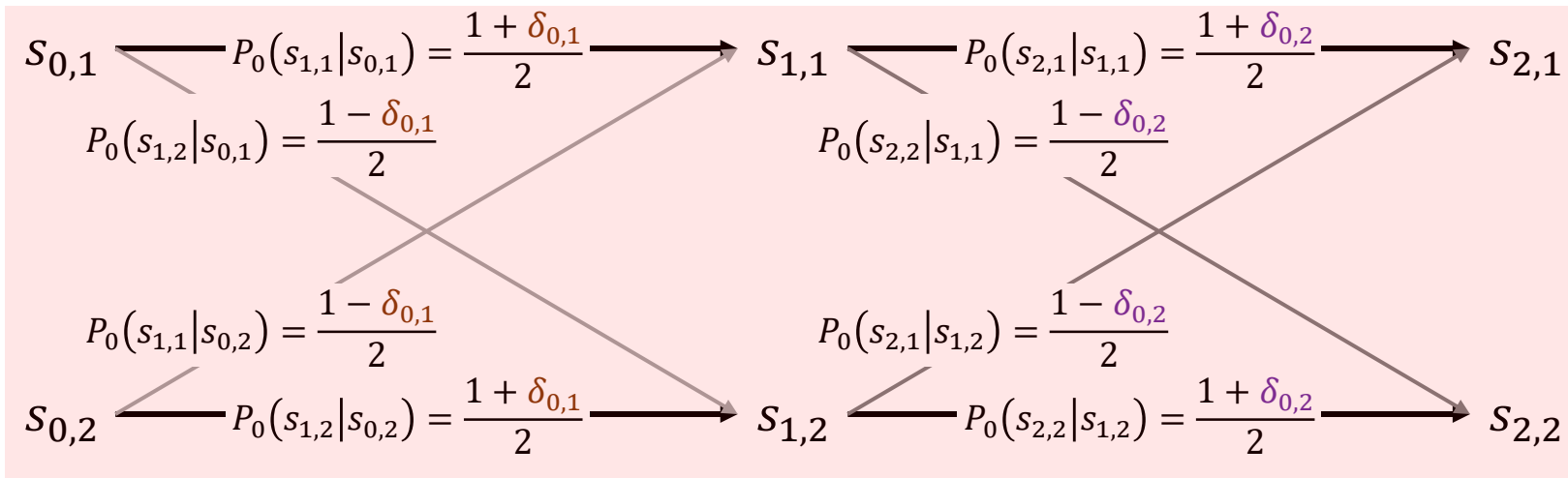
With random trajectory (S_0, S_1, S_2) sampled from $\mathcal{D}_1$, suppose the reasoning process is updated as

$$\delta_{1,1} = \frac{\delta_{0,1} + \delta_{0,2}}{1 + \delta_{0,1}\delta_{0,2}}$$

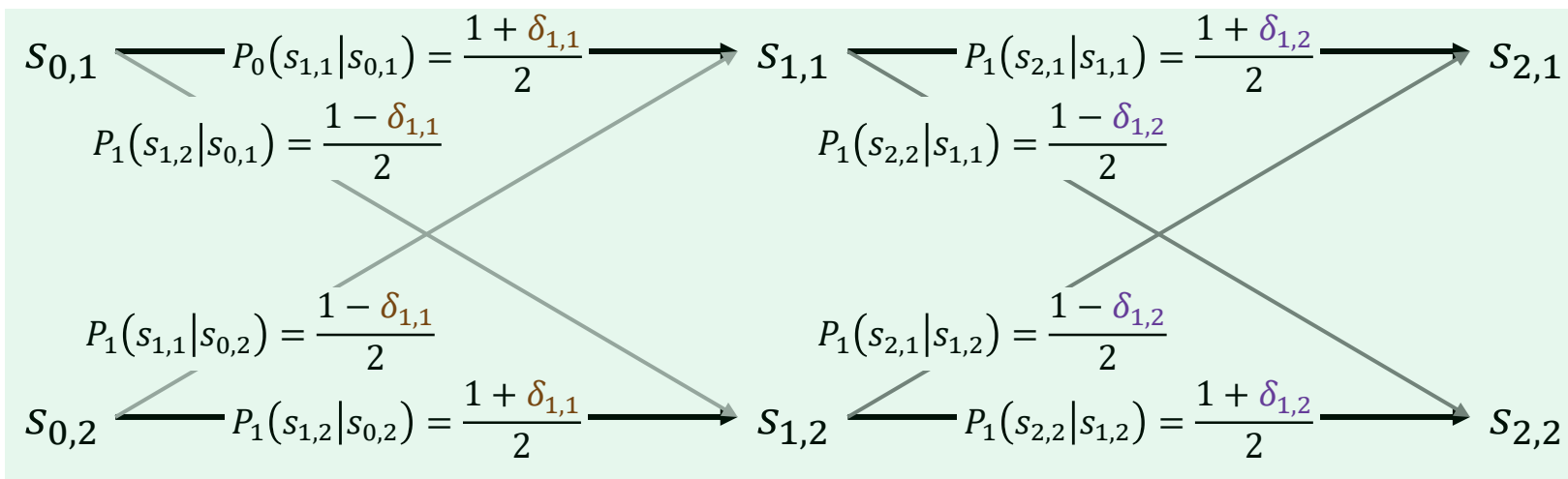
$$\delta_{1,2} = \frac{\delta_{0,1} + \delta_{0,2}}{1 + \delta_{0,1}\delta_{0,2}}$$



Toy Example



Fine tune with trajectories which yields correct answer



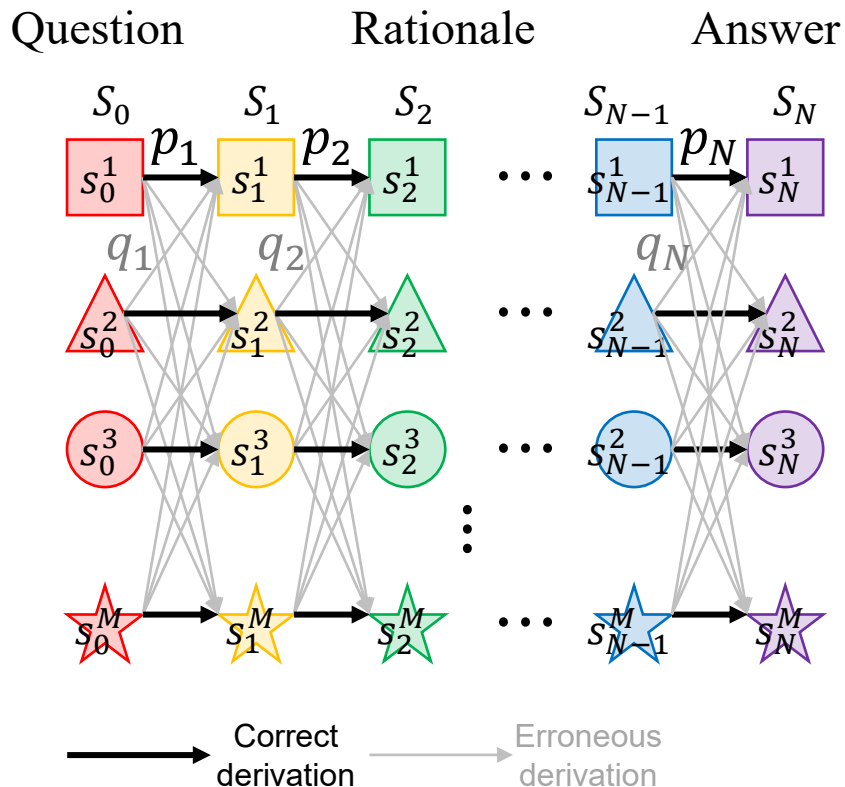
$$\delta_{1,1} = \frac{\delta_{0,1} + \delta_{0,2}}{1 + \delta_{0,1}\delta_{0,2}} > \frac{\delta_{0,1}(1 + \delta_{0,1}\delta_{0,2})}{1 + \delta_{0,1}\delta_{0,2}} = \delta_{0,1}$$

$$\delta_{1,2} = \frac{\delta_{0,1} + \delta_{0,2}}{1 + \delta_{0,1}\delta_{0,2}} > \frac{\delta_{0,2}(1 + \delta_{0,1}\delta_{0,2})}{1 + \delta_{0,1}\delta_{0,2}} = \delta_{0,2}$$

Self Taught Process – A simplified model



We consider a simplified model, where the chain-of-thought is a Markov process with a fixed N steps of derivations



Suppose at step n the prob for correct/incorrect rationale are specified by

$$\mathbb{P} \left[S_{n+1}^{(\text{old})} = s_{n+1}^i | S_n^{(\text{old})} = s_n^i \right] = p_n^{(\text{old})} = \frac{1 + (M - 1)\delta_n^{(\text{old})}}{M}$$

$$\mathbb{P} \left[S_{n+1}^{(\text{old})} = s_{n+1}^j | S_n^{(\text{old})} = s_n^i \right] = q_n^{(\text{old})} = \frac{1 - \delta_n^{(\text{old})}}{M}$$

That is, correct rationale is slightly better than random guess. Then the generated rationales with the correct final answer is a Markov process with

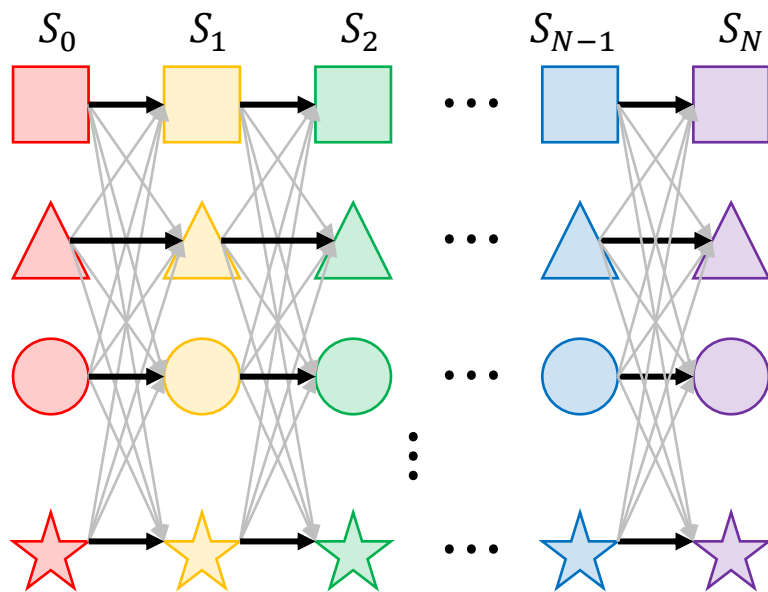
$$\mathbb{P} \left[S_{n+1}^{(\text{new})} = s_{n+1}^i | S_n^{(\text{new})} = s_n^i \right] = p_n^{(\text{new})} = \frac{1 + (M - 1)\delta_n^{(\text{new})}}{M}$$

$$\mathbb{P} \left[S_{n+1}^{(\text{new})} = s_{n+1}^j | S_n^{(\text{new})} = s_n^i \right] = q_n^{(\text{new})} = \frac{1 - \delta_n^{(\text{new})}}{M}$$

It turns out that $\delta_n^{(\text{new})} > \delta_n^{(\text{old})}$ under mild conditions, and we may characterize how fast δ_n converges to 1 (that corresponds to perfect reasoning)

General Case

Question Rationale Answer



Let $S = (S_n)_{n=0}^{N-1}$ be a non-homogeneous Markov sequence with state space $\mathbb{S} = [1, M]$, initial distribution μ , and transition matrix $P_n \in \mathbb{R}^{M \times M}$. More specifically,
 $S_0 \sim \mu, [P_n]_{i,j} = \mathbb{P}[S_{n+1} = j | S_n = i]$.

Proposition.

Consider $T = \{(s_n)_{n=0}^{N-1} \in \mathbb{S}^N : s_0 = s_{N-1}\}$ and
 $[\tilde{P}_n]_{i,j} = \mathbb{P}[S_{n+1} = j | S \in T, S_n = i]$

If μ is uniform, and that

$$[P_n]_{i,i} = \frac{1 + (M-1)\delta_n}{M}, \quad [P_n]_{i,j} = \frac{1 - \delta_n}{M} \quad (i \neq j)$$

Then

$$[\tilde{P}_n]_{i,i} = \frac{1 + (M-1)\tilde{\delta}_n}{M}, \quad [\tilde{P}_n]_{i,j} = \frac{1 - \tilde{\delta}_n}{M} \quad (i \neq j)$$

where $\tilde{\epsilon}_n = 1 - \tilde{\delta}_n$ and $\epsilon_n = 1 - \delta_n$ satisfy

$$\tilde{\epsilon}_n = \frac{1 - \prod_{k \neq n} \delta_k}{1 + (M-1)} \epsilon_n$$

Furthermore, denote $\vartheta = 1 - \prod_{k=0}^{N-1} \delta_k$, one has

$$\sum_{k=0}^{N-1} \tilde{\epsilon}_k \leq \frac{\vartheta}{M - (M-1)\vartheta} \sum_{k=0}^{N-1} \epsilon_k$$

$\tilde{\theta}$ θ

Question: If fine-tuning is performed many iterations, will the reasoning model improves towards perfection?

Remark:

- If the reasoning model is better than random guess ($\delta_n > 0$), then fine-tuning improves reasoning ($\tilde{\epsilon}_n > \epsilon_n$, namely $\tilde{\delta}_n > \delta_n$)
- Both θ and ϑ measure how "imperfect" the whole reasoning model is, which decreases during fine-tuning

Convergence rate analysis



Recall that during one fine-tuning iteration, one has

$$\sum_{k=0}^{N-1} \tilde{\epsilon}_k \leq \frac{\vartheta}{M - (M-1)\vartheta} \sum_{k=0}^{N-1} \epsilon_k$$

$$\vartheta = 1 - \prod_{k=0}^{N-1} \delta_k \leq \min \left\{ 1, \sum_{k=0}^{N-1} (1 - \delta_k) \right\} = \min\{1, \theta\}$$

Now suppose we fine tune for many iterations, and denote θ_t, ϑ_t as the “imperfection” of the reasoning model after iteration t , we have the following recurrence:

$$\vartheta_t \leq \theta_t$$

$$\theta_{t+1} \leq \frac{\vartheta_t}{M - (M-1)\vartheta_t} \theta_t$$

The next proposition shows how fast θ_t and ϑ_t converges towards zero (which corresponds to perfect reasoning).

Proposition.

Let $(\theta_t)_{t=0}^{\infty}$ and $(\vartheta_t)_{t=0}^{\infty}$ be two non-negative (not strictly) decreasing sequences satisfying $\vartheta_0 \in (0,1)$, $\vartheta_t \leq \theta_t$, and

$$\theta_{t+1} \leq \frac{\vartheta_t}{M - (M-1)\vartheta_t} \theta_t$$

where $M \in \mathbb{N}$. Denote $\gamma = \frac{\vartheta_0}{M - (M-1)\vartheta_0} \in (0,1)$, then

a) $\theta_t \leq \gamma^t \theta_0$. That is, for $\epsilon > 0$, holds

$$\theta_t \leq \epsilon \quad \forall t \geq \max \left\{ \frac{\log(\theta_0/\epsilon)}{\log(1/\gamma)}, 0 \right\}$$

b) If $\theta_0 < \frac{M}{M+1}$, then

$$\theta_t \leq \frac{M(\theta_0)^{2^t}}{M(\theta_0)^{2^t} + (M(1 - \theta_0))^{2^t}}$$

Furthermore, for $\epsilon \in (0, \frac{M}{M+1})$, holds

$$\theta_t \leq \epsilon \quad \forall t \geq \max \left\{ \log_2 \left(\frac{\log M + \log \frac{1-\epsilon}{\epsilon}}{\log M + \log \frac{1-\theta_0}{\theta_0}} \right), 0 \right\}$$

c) For $\epsilon \in (0, \frac{M}{M+1})$, holds

$$\theta_t \leq \epsilon \quad \forall t \geq \left\lceil \frac{\log \frac{M+1}{M\gamma} + \log \theta_0}{\log \left(\frac{1}{\gamma} \right)} \right\rceil_+ + \left\lceil \log_2 \left(\frac{\log M + \log \frac{1-\epsilon}{\epsilon}}{\log \frac{(M+1) - M\gamma}{\gamma}} \right) \right\rceil_+$$

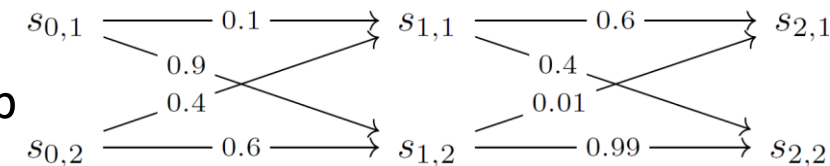
Brief summary and future work

■ Summary:

- This work provides a framework to understand why STaR can achieve policy improvement and convergence toward the optimal policy, despite the wrong reasoning trajectories being included in fine-tuning.

■ Future work:

- Relax the uniformity of $\delta_{t,n}$ within a Reasoning Step
- Multiple Ground-Truth Reasoning Trajectories
- Flexible number of reasoning steps (N) and states (M)
- Relax the Markov Properties of State Transitions



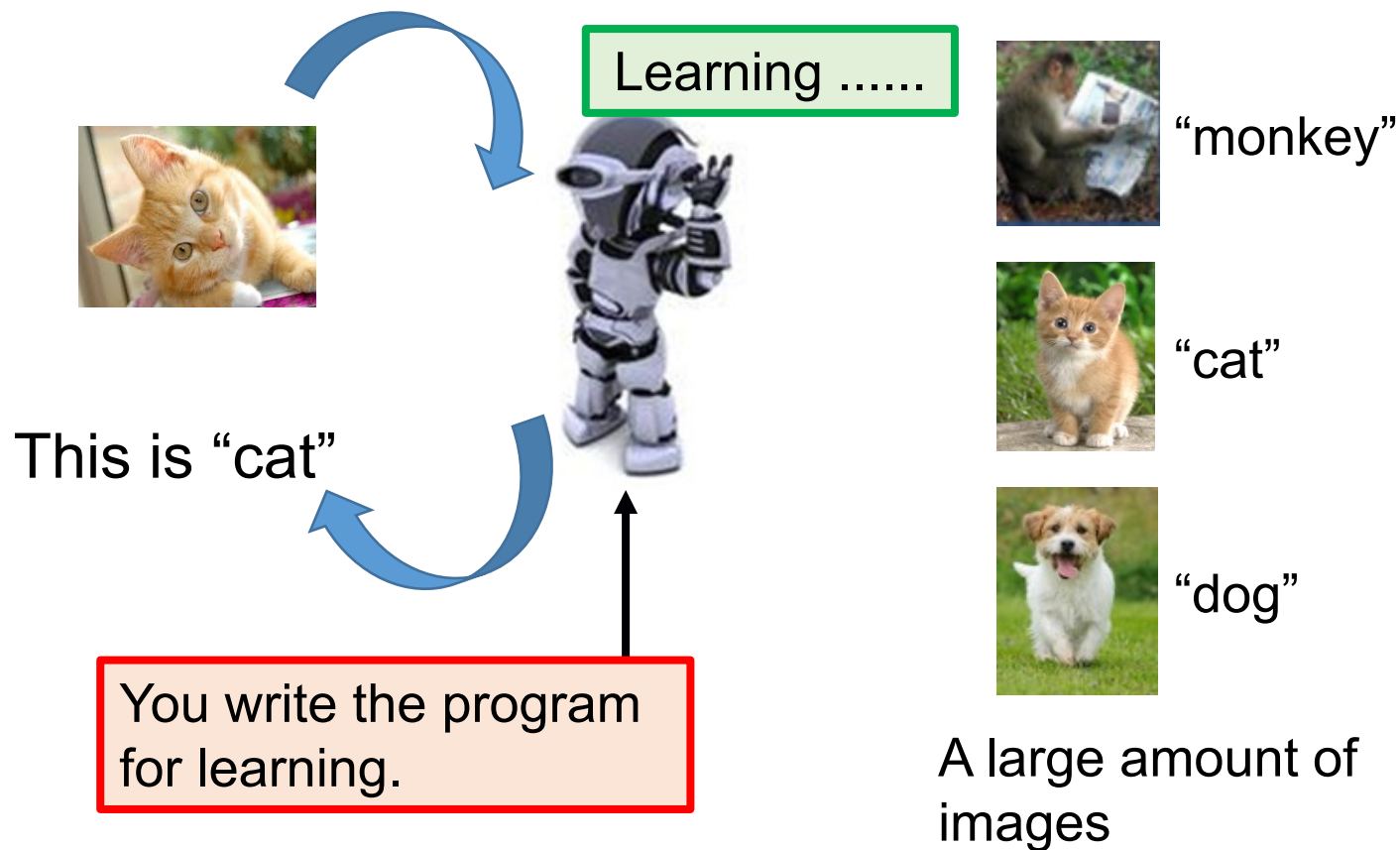
In LLMs, the next state S_n not only depends on S_{n-1} , but also on the history

$$S_0, \dots, S_{n-1}. \quad s_0 = 3 * 2 + 5 * 4 \Rightarrow s_1 = 6 + 5 * 4 \Rightarrow s_2 = 6 + 20 \Rightarrow s_3 = 26,$$

$$s_0 = 3 * 2 + 5 * 4 \Rightarrow s'_1 = 3 * 2 + 20 \Rightarrow s_2 = 6 + 20 \Rightarrow s_3 = 26$$

Gradient Descent with Adversarial Corruption

What is Machine Learning?



Machine Learning ≈ Looking for a Function

Speech Recognition

$g(\text{[audio waveform]}) = \text{"How are you"}$

Image Recognition

$g(\text{[cat image]}) = \text{"Cat"}$

Playing Go

$g(\text{[Go board state]}) = \text{"5-5" (next move)}$

Dialogue System

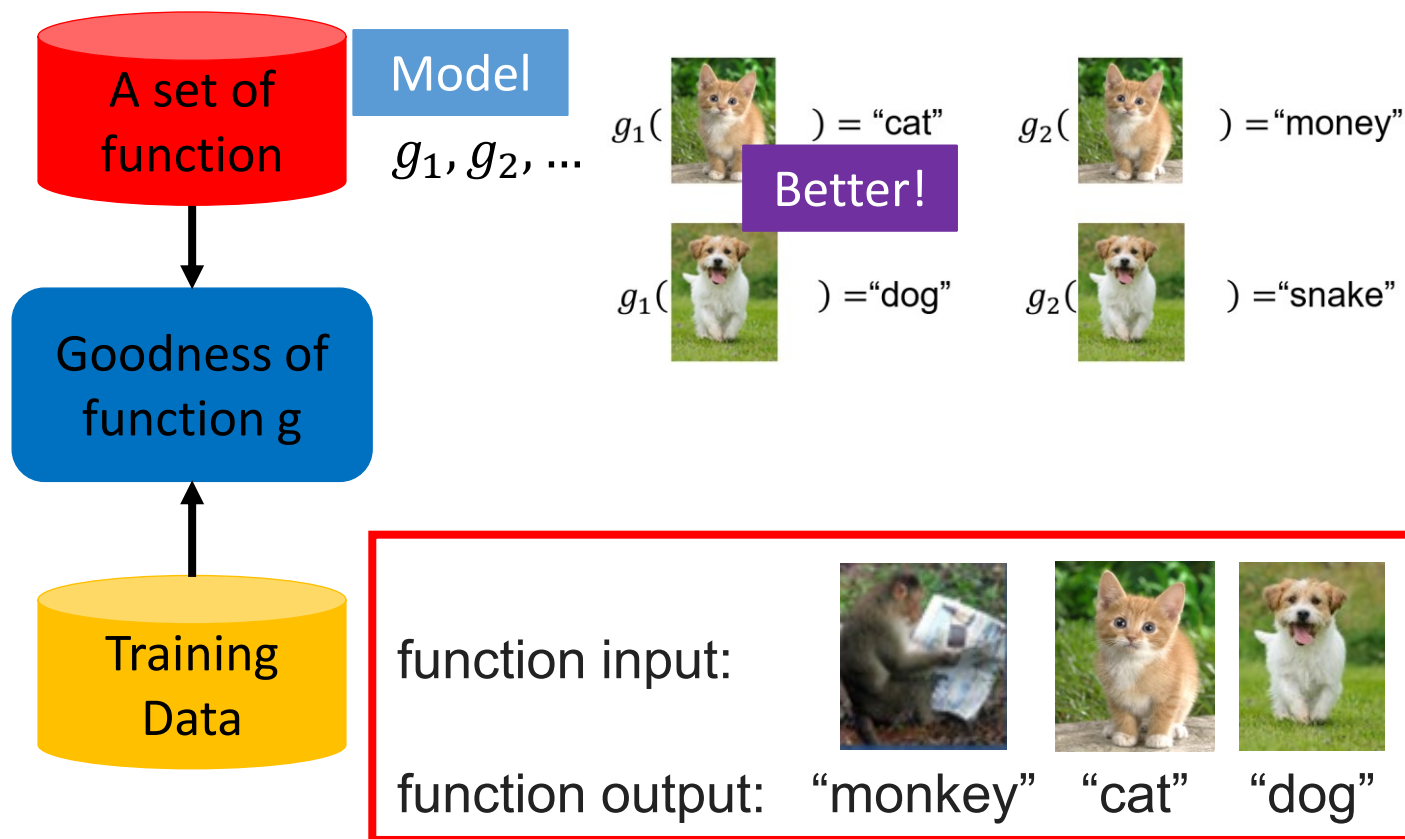
$g(\text{"Hi" (what the user said)}) = \text{"Hello" (system response)}$



Framework

Image Recognition:

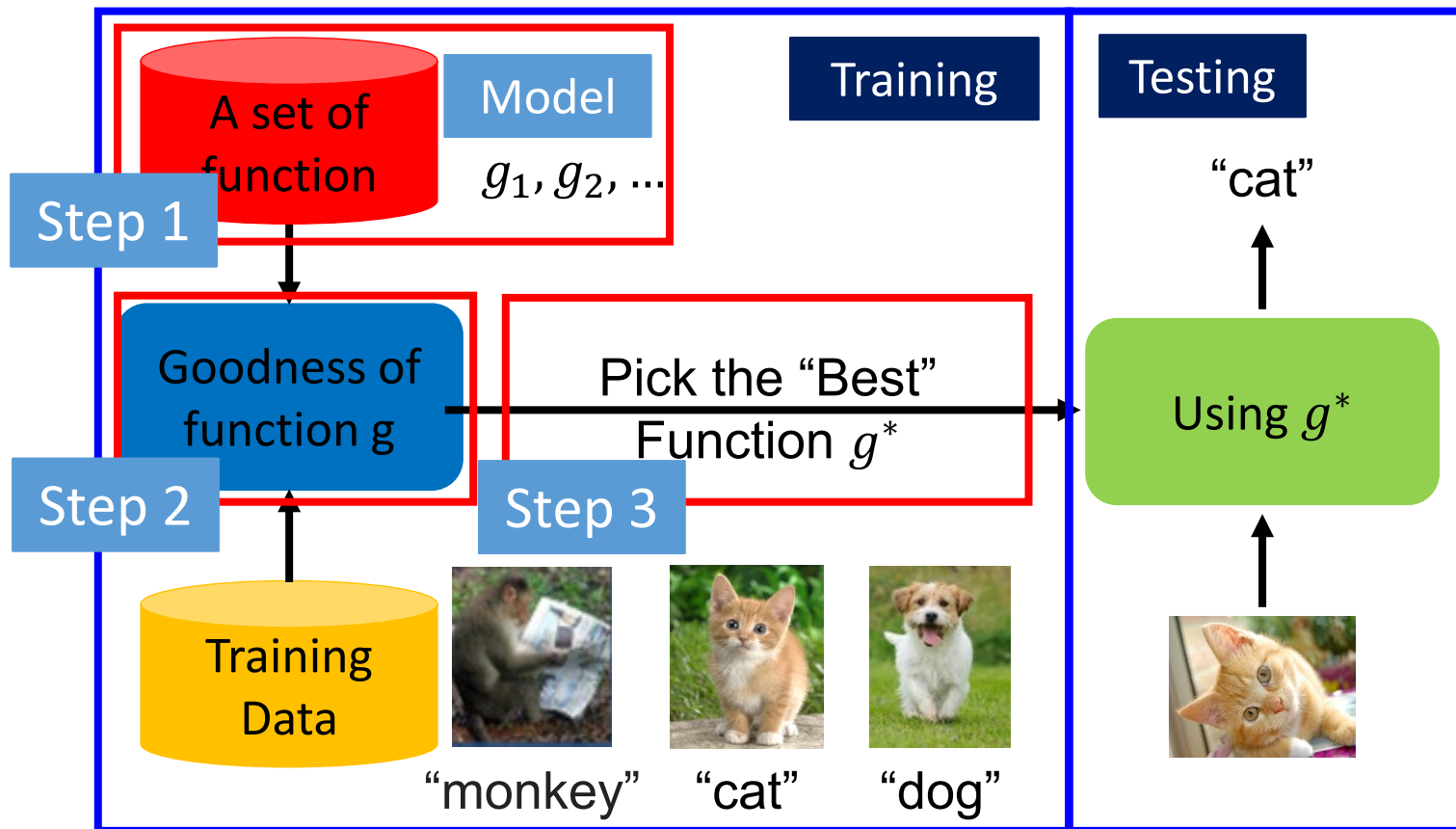
$$g(\text{img_cat}) = \text{"cat"}$$



Framework

Image Recognition:

$g(\text{Image of a cat}) = \text{"cat"}$



Gradient Descent

- Machine learning models are often learned through minimizing some loss function f , namely to solve

$$\mathbf{x}^* \in \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$$

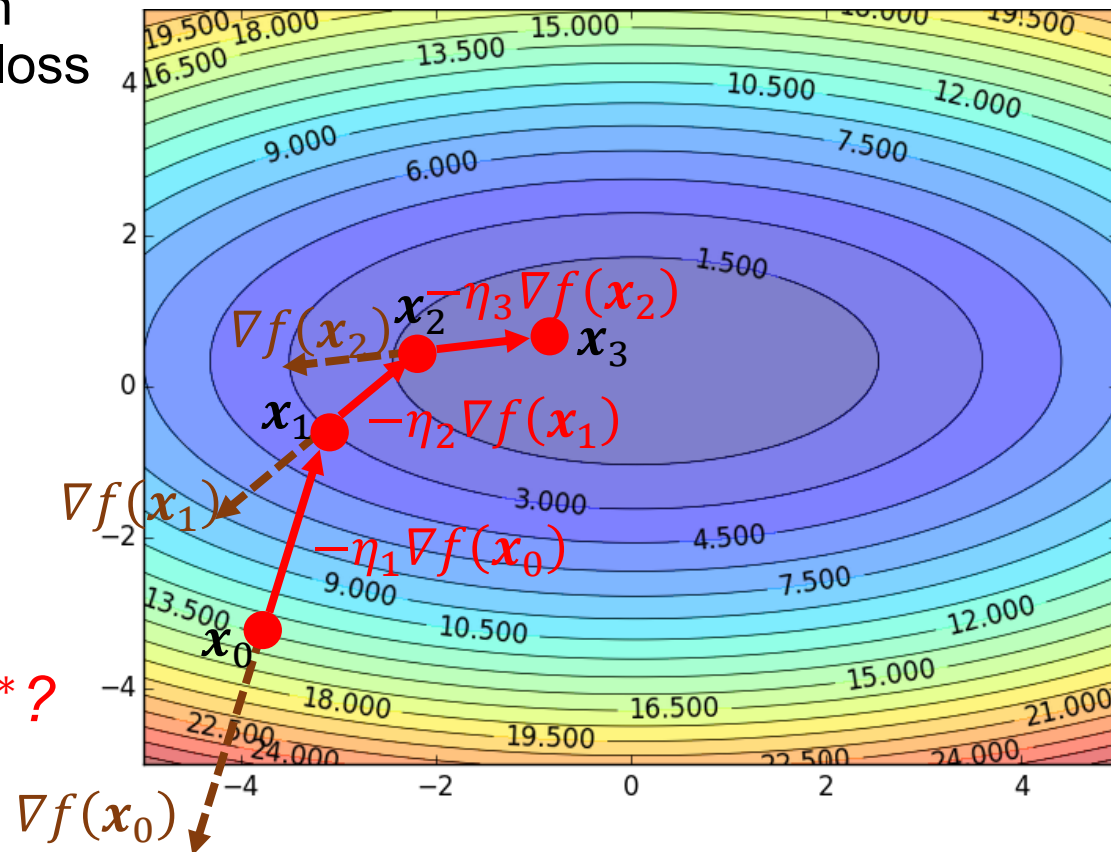
- Gradient descent (GD):

1. Randomly choose $\mathbf{x}_0$

2. Iteratively update

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k \nabla f(\mathbf{x}_{k-1})$$

How fast does $\mathbf{x}_k$ approaches $\mathbf{x}^$?*



Strong convexity and smoothness

■ Definition (Strong convexity)

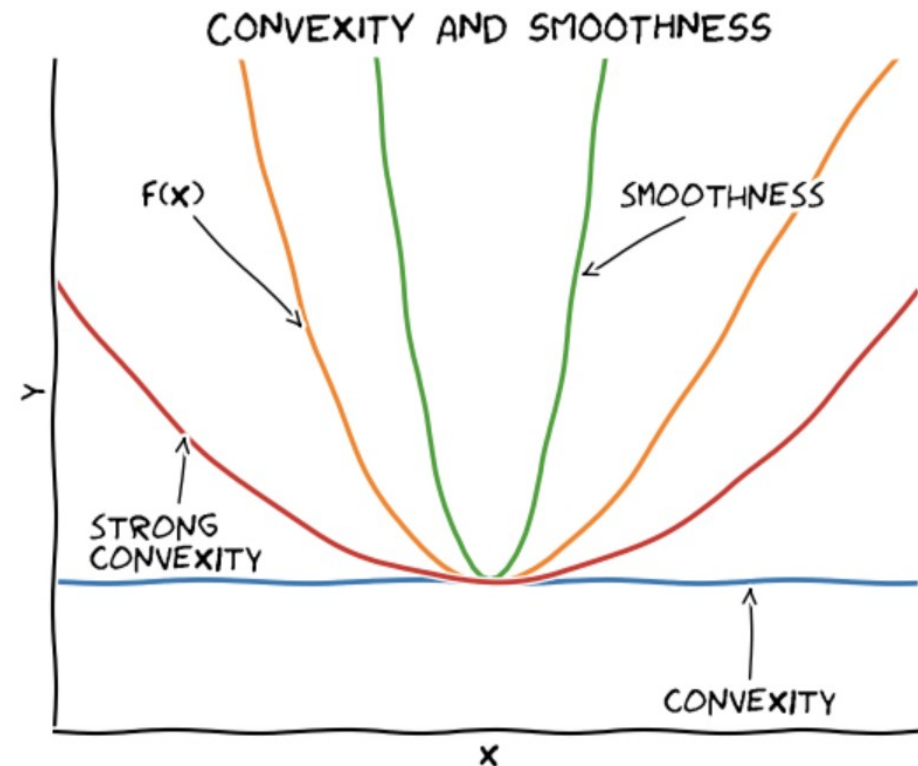
We say a function f is strongly convex with parameter μ , if

$$f(x) \geq f(x') + \langle \nabla f(x'), x - x' \rangle + \frac{\mu}{2} \|x - x'\|^2$$

■ Definition (Smoothness)

We say a function f is smooth with parameter β , if

$$f(x) \leq f(x') + \langle \nabla f(x'), x - x' \rangle + \frac{\beta}{2} \|x - x'\|^2$$



Linear convergence rate for smooth strongly convex functions

- **Theorem (Well known):** If f is μ -strongly convex and β -smooth, then GD with learning rate $\eta_k \leq 1/\beta$ yields

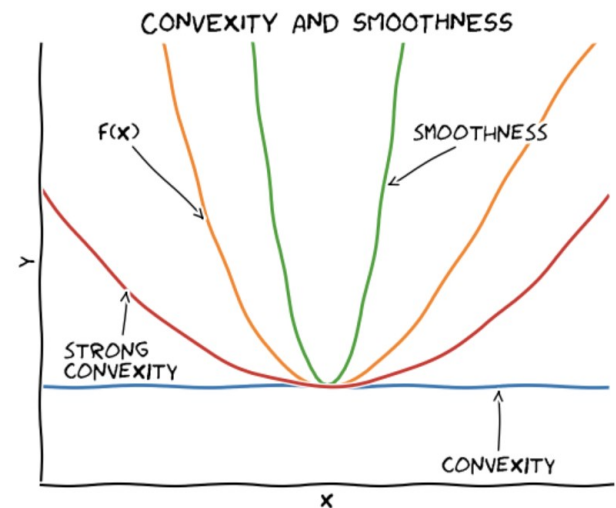
$$\|x_k - x^*\|^2 \leq (1 - \mu\eta_k)\|x_{k-1} - x^*\|^2$$

where we recall $x_k = x_{k-1} - \eta_k \nabla f(x_{k-1})$. In particular, take $\eta_k = 1/\beta$ yields

$$\|x_k - x^*\|^2 \leq (1 - \mu/\beta)\|x_{k-1} - x^*\|^2$$

In other words,

$$\|x_k - x^*\| \leq \sqrt{1 - \mu/\beta}^k \|x_0 - x^*\|$$



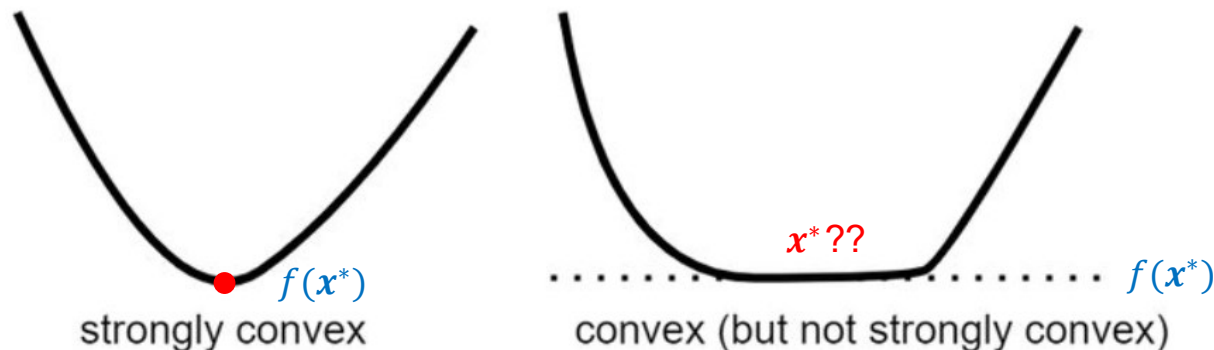
Relax the strongly convex condition

- By strong convexity + smoothness, we can get something like

$$\|x_k - x^*\| \leq r \|x_{k-1} - x^*\|$$

for some fixed $r < 1$ (which depends on μ and β)

- Can the strong convexity condition be relaxed?
 - If f is not strongly convex, bounding $\|x_k - x^*\|$ maybe meaningless, as x^* may not be unique.
 - Instead of bounding $\|x_k - x^*\|$, one seeks to bound $f(x_k) - f(x^*)$.



Polyak-Lojasiewicz condition

■ Definition (PL condition)

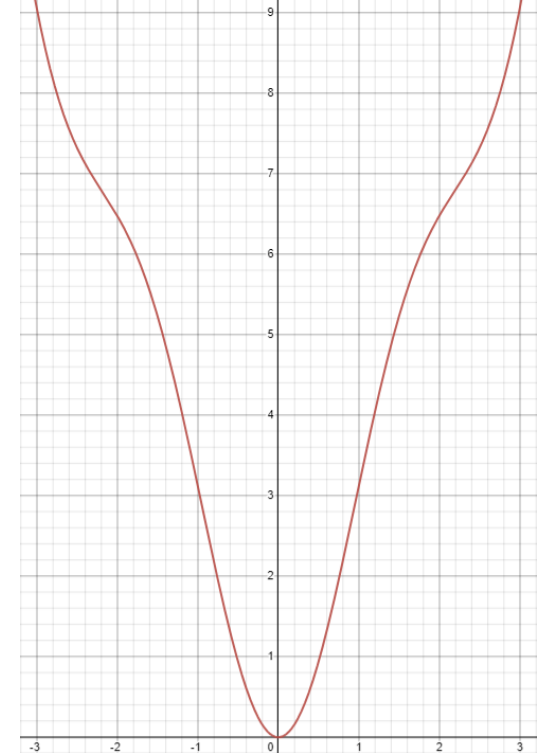
A function f is said to satisfy the Polyak-Lojasiewicz (PL) condition with parameter α , if

$$\|\nabla f(\mathbf{x})\|^2 \geq \alpha(f(\mathbf{x}) - f(\mathbf{x}^*))$$

■ Remark

- Strong convexity implies PL.
 - ✓ If f is μ -strongly convex, then it is (2μ) -PL. (Well known)
- PL condition **does not** imply convex.
- PL + smoothness implies linear convergence rate for GD (with properly chosen learning rate)

$$f(x) = x^2 + 3 \sin^2(x)$$



Linear convergence rate for smooth PL functions

- **Theorem (Polyak 1963):** If f is α -PL and β -smooth, then GD converges at linear rate, namely

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq (1 - \alpha\eta_k(1 - \beta\eta_k/2))(f(\mathbf{x}_{k-1}) - f(\mathbf{x}^*))$$

where $\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k \nabla f(\mathbf{x}_{k-1})$. In particular, take $\eta_k = 1/(2\beta)$ yields

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq (1 - \alpha/(2\beta))(f(\mathbf{x}_{k-1}) - f(\mathbf{x}^*))$$

In other words,

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \left(1 - \frac{\alpha}{2\beta}\right)^k (f(\mathbf{x}_0) - f(\mathbf{x}^*))$$

Stochastic Gradient Descent

- Stochastic gradient decent (SGD) aims to solve the stochastic optimization problem:

$$\mathbf{x}^* \in \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}) = \mathbb{E}[f_i(\mathbf{x})]$$

where the expectation is taken w.r.t. i . A typical scenario is to optimize finite sums

$$f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{x})$$

- SGD uses the iteration

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k \nabla f_{i_k}(\mathbf{x}_{k-1})$$

where i_k is a sample over $1, \dots, N$, such that

$$\mathbb{E}[\nabla f_{i_k}(\mathbf{x}_{k-1})] = \nabla f(\mathbf{x}_{k-1})$$

Stochastic Gradient Descent

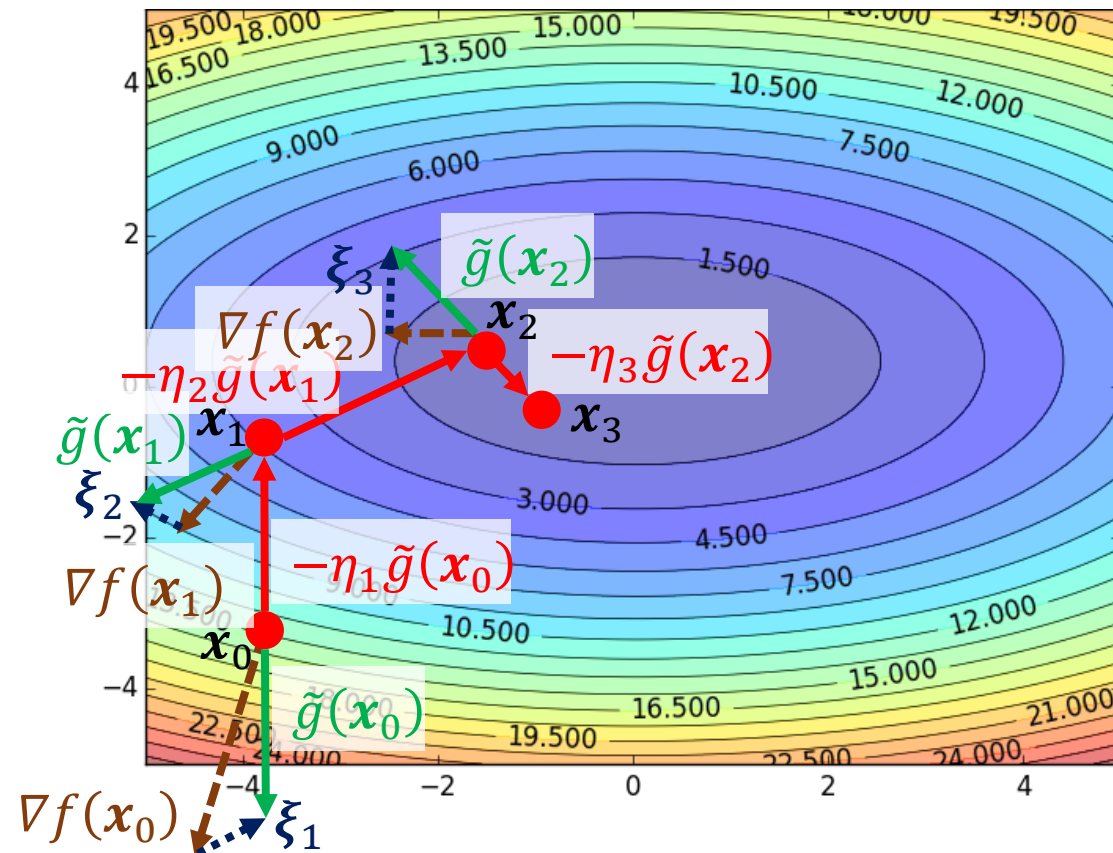
- In GD, the update rule is

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k \nabla f(\mathbf{x}_{k-1})$$

- In SGD, the update rule becomes

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k \tilde{\mathbf{g}}(\mathbf{x}_{k-1})$$

$$\tilde{\mathbf{g}}(\mathbf{x}_{k-1}) = \nabla f(\mathbf{x}_{k-1}) + \boldsymbol{\xi}_k$$



SGD can achieve $O(1/k)$ convergence rate

- **Theorem (Karimi 2016)** If f is α -PL and β -smooth, and suppose that $\mathbb{E}[\|\nabla f_i(\mathbf{x}_k)\|^2] \leq \gamma^2$ for all k and some γ . Then SGD with learning rate

$$\eta_k = \frac{2k - 1}{\alpha k^2}$$

yields a convergence rate of

$$\mathbb{E}[f(\mathbf{x}_k) - f(\mathbf{x}^*)] \leq \frac{2\beta\gamma^2}{\alpha^2 k}$$

Hamed Karimi, Julie Nutini, and Mark Schmidt. 2016. **Linear Convergence of Gradient and Proximal-Gradient Methods Under the Polyak-Łojasiewicz Condition**. In *European Conference on Machine Learning and Knowledge Discovery in Databases* - Volume 9851 (ECML PKDD 2016). Springer-Verlag, Berlin, Heidelberg, 795–811. https://doi.org/10.1007/978-3-319-46128-1_50

Adversarial Gradient Descent (Adv-GD)

- In GD, the update rule is

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k \nabla f(\mathbf{x}_{k-1})$$

- In SGD, the update rule becomes

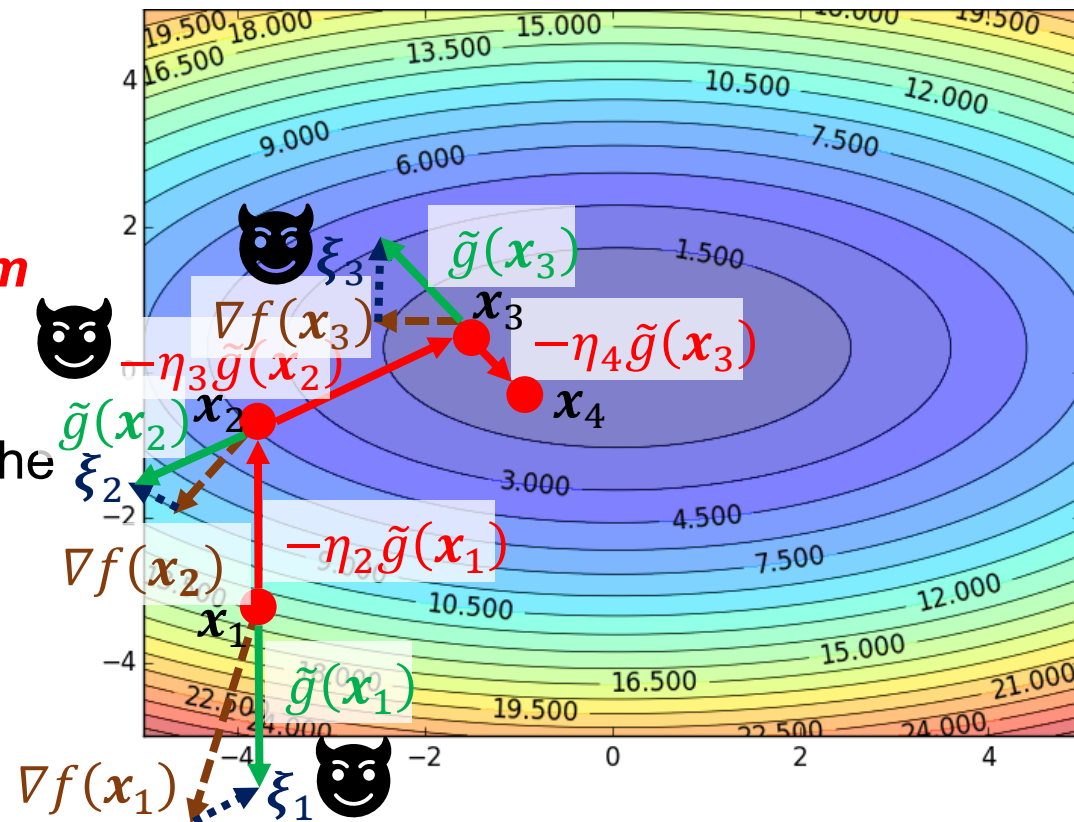
$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k (\nabla f(\mathbf{x}_{k-1}) + \xi_k)$$

where ξ_k is some zero mean **random** noise.

- In Adv-GD, an adversary tries to ruin GD by adding some perturbation to the gradient. Namely, the update rule is

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \eta_k (\nabla f(\mathbf{x}_{k-1}) + \xi_k)$$

where ξ_k is added by **adversary**.



Minimax error bounds under Adv-DG framework

- **Question:** Can we design learning rate sequence $(\eta_k)_{k=1}^{\infty}$ so that $f(\mathbf{x}_n) - f(\mathbf{x}^*)$ diminishes (fast) under adversarial setting?
 - Recall that $\mathbf{x}_n$ depends on $\eta_{1:n}$ and $\xi_{1:n}$.
 - With no constraint, the adversarial corruptions can set $(\mathbf{x}_k)_{k=1}^{\infty}$ to any sequence in $\mathbb{R}^d$. (Not interesting)
 - Assume the adversary has a budget constraint:
$$\|(\|\xi_k\|)_{k=1}^n\|_{\ell_p} = \sqrt[p]{\|\xi_1\|^p + \cdots + \|\xi_n\|^p} \leq b(n)$$

We want to design learning rate sequence $(\eta_k)_{k=1}^{\infty}$ that minimizes

$$r_n = \sup_{\xi_{1:n}: \|(\|\xi_k\|)_{k=1}^n\|_{\ell_p} \leq b(n)} (f(\mathbf{x}_n) - f(\mathbf{x}^*))$$

- **Theorem:** If f is α -PL and β -smooth, and suppose that $\|\nabla f(\mathbf{x}_k)\| \leq \gamma$ for all k and some γ . Let

$$\eta_k = \frac{2}{\alpha} \left(1 - \left(\frac{k}{k+1} \right)^\omega \right) \wedge \frac{1}{\beta}$$

for a fixed $\omega \geq 1$. Then under the Adv-GD framework where the adversary has budget constraint $\|(\|\xi_k\|)_{k=1}^n\|_{\ell_p} \leq b(n)$, we have

$$r_n \leq A_n(f(\mathbf{x}_0) - f(\mathbf{x}^*)) + B_{p,n}b(n) + C_{p,n}(b(n))^2$$

where $A_n \in O(n^{-\omega})$, $B_{p,n} \in O(n^{-1/p})$, and

$$\begin{cases} C_{p,n} \in O(n^{-(\omega \wedge 2)}) & \text{when } 1 \leq p \leq 2 \text{ and } \omega \geq 1 \\ C_{p,n} \in O(n^{-\omega}) & \text{when } p > 2 \text{ and } 1 \leq \omega < 1 + 2/p \\ C_{p,n} \in O(n^{-\omega}(\log n)^{1-2/p}) & \text{when } p > 2 \text{ and } \omega = 1 + 2/p \\ C_{p,n} \in O(n^{-1-2/p}) & \text{when } p > 2 \text{ and } \omega > 1 + 2/p \end{cases}$$

- **Example:** For the case $p = 1$, namely $\|\xi_1\| + \dots + \|\xi_n\| \leq b(n)$, take $\omega = 2$ yields

$$\eta_k = \frac{2}{\alpha} \left(1 - \left(\frac{k}{k+1} \right)^2 \right) \wedge \frac{1}{\beta} \sim \frac{1}{k}, \quad r_n \in O \left(\frac{f(\mathbf{x}_0) - f(\mathbf{x}^*)}{n^2} + \frac{b(n)}{n} + \frac{(b(n))^2}{n^2} \right)$$



Brief summary and future work

- We introduce Adv-GD framework, in which gradients are corrupted by an adversary with l_p -norm constrained budget.
- We provided finite time upper bounds on the worst-case optimization error, for the family of smooth PL functions.
- We show that the robustness of GD algorithm can be achieved via a proper choice of learning rate sequence.
- Future works:
 - Lower bound for the error
 - Optimal adversarial corruption
 - Extend to SGD where the gradient is corrupted by both random noise and adversary corruption.

Linear bandit upper confidence bound

K-armed bandit problem

- We have K decisions (the “arms”), where when we play arm $a \in \llbracket 1, K \rrbracket$ we obtain a random reward with mean $\mu_a \in [0, 1]$.
- Every iteration t , the learner will pick an arm $I_t \in \llbracket 1, K \rrbracket$. The cumulative regret is

$$R_T = T \max_a \mu_a - \sum_{t=1}^T \mu_{I_t}$$

- Can be regarded as a MDP with one state (casino) and K actions (arms)



Expected
Reward

μ_1

μ_2

μ_K

Upper confidence bound (UCB) algorithm

- First play each arm once.

- In iteration t :

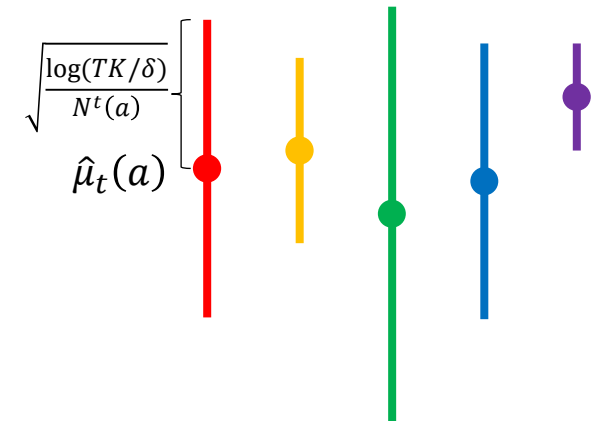
- maintain counts and empirical mean for each arm:

$$N^t(a) = \sum_{i=1}^t 1\{I_i = a\}, \hat{\mu}_t(a) = \frac{1}{N^t(a)} \sum_{i=1}^t 1\{I_i = a\} r_i$$

From which the UCB is computed as

$$\hat{\mu}_t(a) + \sqrt{\frac{\log(TK/\delta)}{N^t(a)}}$$

- Play the arm with the highest UCB.



Algorithm 1 UCB

for $t \leftarrow 1$ to K **do**

$I_t \leftarrow t$

 Play arm I_t and receive reward r_t

end for

for $t \leftarrow K + 1$ to T **do**

$I_t \leftarrow \arg \max_{a \in [1, K]} \left(\hat{\mu}^{t-1}(a) + \sqrt{\frac{\log(TK/\delta)}{N^{t-1}(a)}} \right)$

 Play arm I_t and receive reward r_t

end for

Linear stochastic bandit

- Assume each bandit is characterized by a feature vector x , for which the expected reward is $\langle x, \mu_* \rangle$. Here $\mu_* \in \mathbb{R}^d$ is fixed but unknown.
 - If there are K arms, take $\mu_* = (\mu_1, \dots, \mu_K)$ and represent the k 'th arm as one-hot vector $(0, \dots, 1, \dots, 0)$ will correspond to reward μ_k .

k 'th element
- In round t ,
 - the learner is given a decision set $D_t \subset \mathbb{R}^d$ from which to choose action $x_t \in D_t$.
 - The learner observes reward $r_t = \langle x_t, \mu_* \rangle + \eta_t$, where $\mu_* \in \mathbb{R}^d$ is fixed but unknown, and η_t is a zero mean random noise.
- **Goal:** Maximize total reward $\sum_{t=1}^T \langle x_t, \mu_* \rangle$. In other words, to minimize regret

$$R_T = \sum_{t=1}^T \langle x_t^*, \mu_* \rangle - \sum_{t=1}^T \langle x_t, \mu_* \rangle = \sum_{t=1}^T \langle x_t^* - x_t, \mu_* \rangle$$

where $x_t^* \in \operatorname{argmax}_{x \in D_t} \langle x, \mu_* \rangle$.

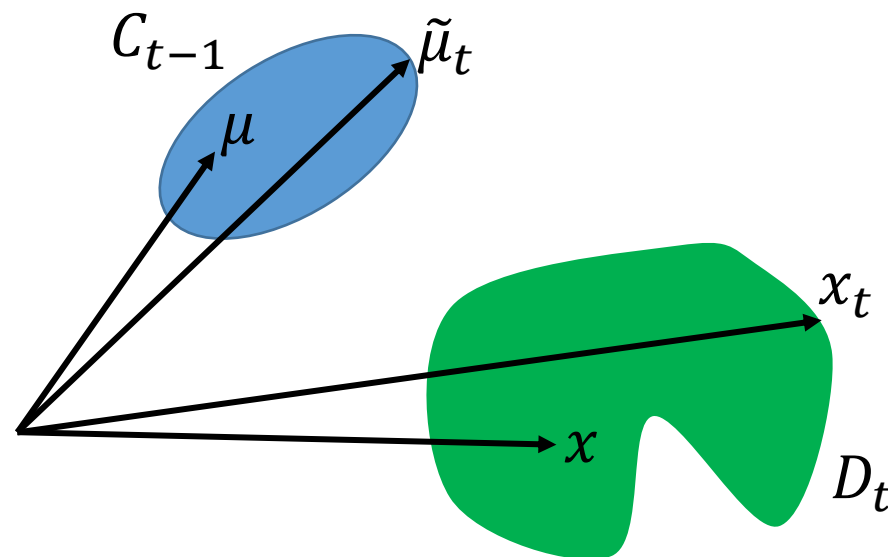


Arm $x \in D_t$
Reward $\langle x, \mu_* \rangle$

Optimism in the face of uncertainty (OFU) principle

- In round t ,
 - Maintains a confidence set $C_{t-1} \subset \mathbb{R}^d$ so that $\mu_* \in C_{t-1}$ “with high probability”.
 - ✓ C_{t-1} is estimated from previously played arms $x_1, \dots, x_{t-1}$ and rewards $r_1, \dots, r_{t-1}$.
 - Chooses an optimistic estimate

$$(x_t, \tilde{\mu}_t) \in \operatorname{argmax}_{x \in D_t, \mu \in C_{t-1}} \langle x, \mu \rangle$$



Algorithm 2 OFUL

```

for  $t \leftarrow 1$  to  $T$  do
   $(x_t, \tilde{\mu}_t) \leftarrow \operatorname{argmax}_{(x, \mu) \in D_t \times C_{t-1}} \langle x, \mu \rangle$ 
  Play  $x_t$  and observe reward  $r_t$ 
  Update  $C_t$ 
end for
  
```

Confidence set

How to estimate the confidence set C_t ?

- Recall that $r_t \approx \langle x_t, \mu_* \rangle$, so we estimate μ_* through ridge regression

$$\begin{aligned}\hat{\mu}_t &= \operatorname{argmin}_{\mu} \left\{ \sum_{t=1}^T (\langle x_t, \mu \rangle - r_t)^2 + \rho \|\mu\|^2 \right\} \\ &= \left(\rho I + \sum_{t=1}^T x_t x_t^\top \right)^{-1} \left(\sum_{t=1}^T r_t x_t \right)\end{aligned}$$

- Theorem[1][2]:** Assume $\|\mu_*\| \leq M$ and that each η_t is conditionally R^2 -sub-gaussian, namely

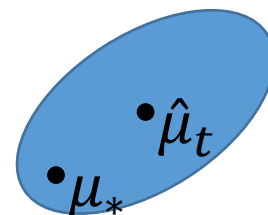
$$\mathbb{E}[e^{\lambda \eta_t} | x_1, \dots, x_t, \eta_1, \dots, \eta_{t-1}] \leq \exp(\lambda^2 R^2 / 2), \forall \lambda \in \mathbb{R}$$

Then for each $\delta > 0$, with probability at least $1 - \delta$, the following holds for all $t \geq 0$:

$$\|\hat{\mu}_t - \mu_*\|_{\Sigma_t} \leq \underbrace{\rho \left\| \Sigma_t^{-\frac{1}{2}} \right\| M + R \sqrt{\log \left(\frac{\det(\Sigma_t)}{\delta^2 \det(\Sigma_0)} \right)}}_{\beta_t}$$

where $\Sigma_t = \rho I + \sum_{i=1}^t x_i x_i^\top$ and denote $\|v\|_{\Sigma} = \sqrt{v^\top \Sigma v}$

$$C_t = \{\mu: \|\mu - \hat{\mu}_t\|_{\Sigma_t} \leq \beta_t\}$$



High probability μ_* lies in C_t for all $t = 0, 1, \dots$

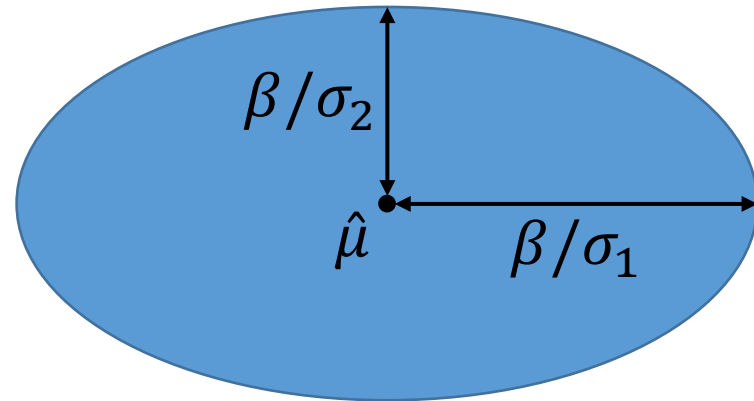
[1] Varsha Dani, Thomas P Hayes, and Sham M Kakade. Stochastic linear optimization under bandit feedback. In COLT, pages 355–366, 2008.

[2] Yasin Abbasi-Yadkori, David Pál, and Csaba Szepesvári. Improved algorithms for linear stochastic bandits. In Advances in Neural Information Processing Systems, pages 2312–2320, 2011.

Confidence set

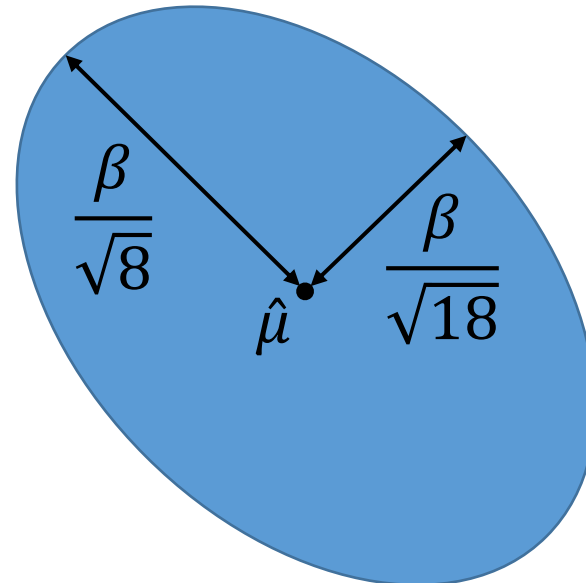
$$\mathcal{C} = \{\mu: \|\mu - \hat{\mu}\|_{\Sigma} \leq \beta\}$$

$$\Sigma = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 13 & 5 \\ 5 & 13 \end{bmatrix} = UDU^{\top}$$

$$U = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, D = \begin{bmatrix} 18 & 0 \\ 0 & 8 \end{bmatrix}$$

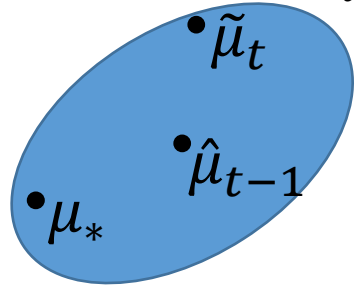


Linear UCB

Algorithm 2 OFUL

for $t \leftarrow 1$ to T do
 $(x_t, \tilde{\mu}_t) \leftarrow \arg \max_{(x, \mu) \in D_t \times C_{t-1}} \langle x, \mu \rangle$
 Play x_t and observe reward r_t
 Update C_t
end for

$$C_{t-1} = \{\mu: \|\mu - \hat{\mu}_{t-1}\|_{\Sigma_{t-1}} \leq \beta_{t-1}\}$$



$$\max_{\mu \in C_{t-1}} \langle x, \mu \rangle = \langle x, \hat{\mu}_{t-1} \rangle + \beta_{t-1} \sqrt{x^\top \Sigma_{t-1}^{-1} x}$$



Algorithm 3 LinUCB

$b_0 \leftarrow 0, \Sigma_0 \leftarrow \rho I$
 for $t \leftarrow 1$ to T do
 $\hat{\mu}_{t-1} \leftarrow \Sigma_{t-1}^{-1} b_{t-1}$
 $\beta_{t-1} \leftarrow R \sqrt{\log \frac{\det(\Sigma_{t-1})}{\delta^2 \det(\Sigma_0)}} + \|\Sigma_{t-1}^{-1/2} \Sigma_0\| M$
 $x_t \leftarrow \arg \max_{x \in D_t} \left(\langle x, \hat{\mu}_{t-1} \rangle + \beta_{t-1} \sqrt{x^\top \Sigma_{t-1}^{-1} x} \right)$
 Play x_t and observe reward r_t
 $\Sigma_t \leftarrow \Sigma_{t-1} + x_t x_t^\top$
 $b_t \leftarrow b_{t-1} + x_t r_t$
end for

$$\hat{\mu}_t = \left(\rho I + \sum_{t=1}^T x_t x_t^\top \right)^{-1} \left(\sum_{t=1}^T r_t x_t \right)$$

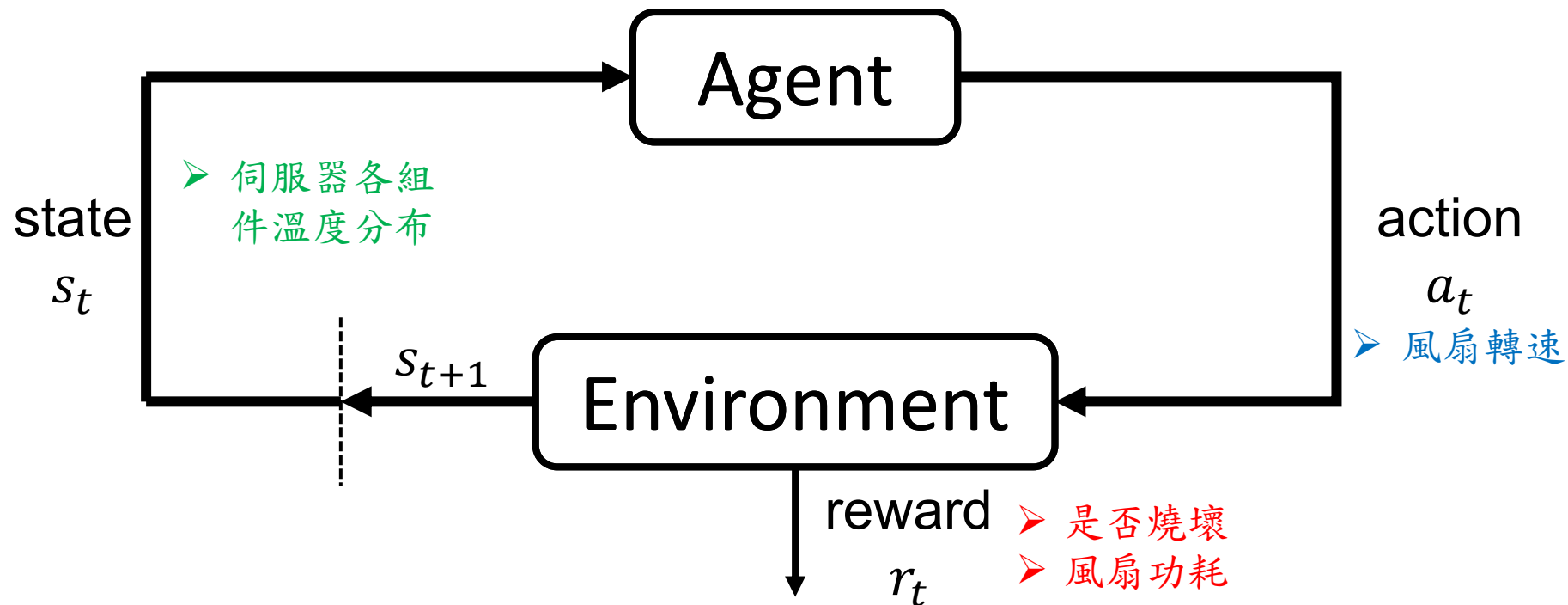
$$\Sigma_t = \rho I + \sum_{i=1}^t x_i x_i^\top$$

Proposition: If $\mu_* \in C_{t-1}$, then

$$\underbrace{\langle x_t^*, \mu_* \rangle - \langle x_t, \mu_* \rangle}_{\text{regret}} \leq 2\beta_{t-1} \sqrt{x_t^\top \Sigma_{t-1}^{-1} x_t}$$

Reinforcement learning

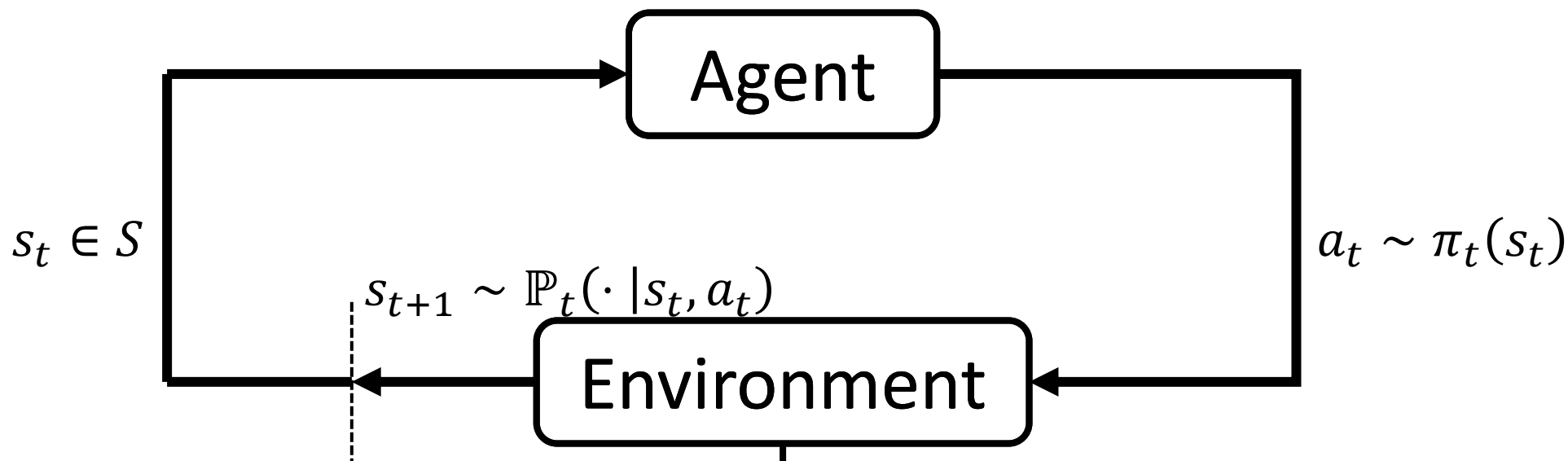
Reinforcement Learning



長期收益最大化：

- 維持理想的溫度
- 降低整體風扇能耗
- 提高應用效率

Episodic Markov Decision Process (MDP)



Episodic MDP is characterized by
 $(S, A, (\mathbb{P}_t)_{t=0}^{H-1}, (\mathbf{r}_t)_{t=0}^{H-1}, H, \mu)$:

S : **State space**

A : **Action space**

$\mathbb{P}_t: S \times A \rightarrow \Delta(S)$: **Transition probability**

$\mathbf{r}_t: S \times A \rightarrow [0, 1]$: **Reward function**

$\mu \in \Delta(S)$: **Initial state distribution**

$H \in \mathbb{N}$: **Horizon**

$\Delta(S)$: Collection of probability distributions on S

$\pi_t: S_t \rightarrow \Delta(A_t)$: **policy** of the agent

$$r_t = \mathbf{r}_t(s_t, a_t)$$

Objective:

Find the optimal policy π^* that maximizes the cumulative rewards $\sum_{t=0}^{H-1} r_t$

Online vs offline RL

- Following a policy π produces sample trajectory (path) $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$
- **Online RL:** Update policy immediately after a path is collected (from previous policy)
- **Offline RL:** Collect a log of paths, then update policy

Reinforcement Learning with Online Interactions



Offline Reinforcement Learning



<https://huggingface.co/learn/deep-rl-course/en/unitbonus3/offline-online>

Value function and Q-function

- With episodic MDP($S, A, (\mathbb{P}_t)_{t=0}^{H-1}, (\mathbf{r}_t)_{t=0}^{H-1}, H, \mu$), following a policy $\pi = (\pi_t)_{t=0}^{H-1}$ produces sample trajectory (path)

$$\tau = (s_0, a_0, r_0, s_1, \dots)$$

where $s_0 \sim \mu$, $a_t \sim \pi_t(\cdot | s_t)$, $r_t = \mathbf{r}_t(s_t, a_t)$

- The **value function** at state s is the expected cumulative reward from following the policy from state s

$$V_h^\pi(s) = \mathbb{E} \left[\sum_{t=h}^{H-1} r_t \mid \pi, s_h = s \right]$$

- The **Q-value function** at state s and action a is the expected cumulative reward from taking action a in state s and then following the policy:

$$Q_h^\pi(s, a) = \mathbb{E} \left[\sum_{t=h}^{H-1} r_t \mid \pi, s_h = s, a_h = a \right]$$

Bellman consistency equations

How to compute V_h^π and Q_h^π ?

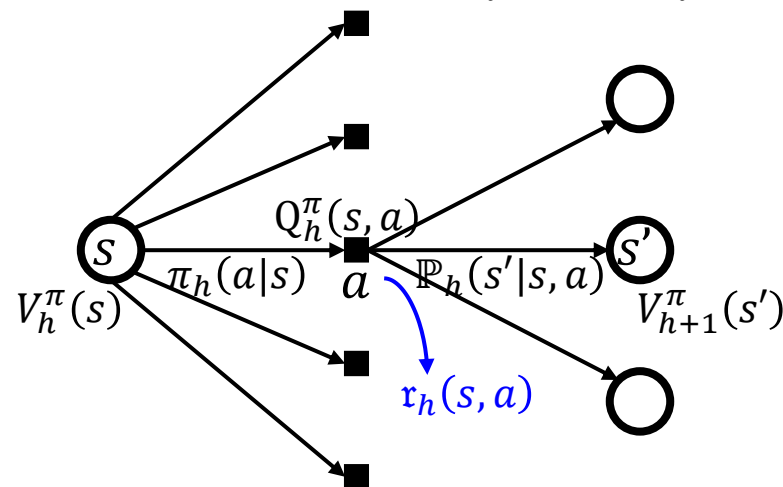
■ **Bellman consistency equation for episodic MDP:**

$$V_h^\pi(s) = \mathbb{E}_{a \sim \pi_h(\cdot|s)} [Q_h^\pi(s, a)]$$

$$Q_h^\pi(s, a) = r_h(s, a) + \mathbb{E}_{s' \sim \mathbb{P}_h(\cdot|s, a)} [V_{h+1}^\pi(s')]$$

Note that $V_H^\pi(s) = 0$, $Q_H^\pi(s, a) = 0$

- If we know $\mathbb{P}_h$ and r_h , we can compute V_h^π and Q_h^π for $h = H - 1, \dots, 0$ through dynamic programming.
- Otherwise, we have to estimate V_h^π and Q_h^π from data.



Bellman optimality equations

How to find optimal policy $\pi^* = (\pi_h^*)_{h=0}^{H-1}$ such that

$$\begin{aligned} V_h^{\pi^*}(s) &\geq V_h^\pi(s) \\ Q_h^{\pi^*}(s, a) &\geq Q_h^\pi(s, a) \end{aligned}$$

for all state s , action a , step h , and policy π ?

■ Bellman optimality equation for episodic MDP:

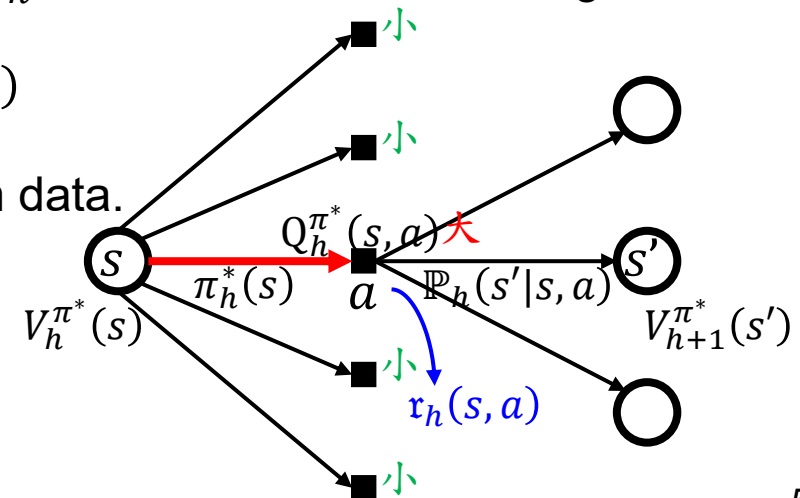
$$\begin{aligned} V_h^{\pi^*}(s) &= \max_{a \in A} Q_h^{\pi^*}(s, a) \\ Q_h^{\pi^*}(s, a) &= r_h(s, a) + \mathbb{E}_{s' \sim \mathbb{P}_h(\cdot | s, a)} [V_{h+1}^{\pi^*}(s')] \end{aligned}$$

Note that $V_H^{\pi^*}(s) = 0$, $Q_H^{\pi^*}(s, a) = 0$

- If we know $\mathbb{P}_h$ and r_h , we can compute $\pi_h^*, V_h^{\pi^*}, Q_h^{\pi^*}$ for $h = H - 1, \dots, 0$ through dynamic programming, where

$$\pi_h^*(s) = \operatorname{argmax}_{a \in A} Q_h^{\pi^*}(s, a)$$

- Otherwise, we have to estimate $\pi^*, V_h^{\pi^*}, Q_h^{\pi^*}$ from data.



Least square value iteration (LSVI)

- In offline setting, collect N trajectories sampled from some behavior policy π
 $(s_0^n, a_0^n, r_0^n, s_1^n, \dots), n = 1, \dots, N$

and assume the optimal Q-function takes linear form:

$$Q_h^{\pi^*}(s, a) = \langle \theta_h^*, \phi(s, a) \rangle$$

where $\phi(s, a) \in \mathbb{R}^d$ is a feature vector. Then

$$\langle \theta_h^*, \phi(s, a) \rangle = r_h(s, a) + \mathbb{E}_{s' \sim \mathbb{P}_h(\cdot | s, a)} [V_{h+1}^{\pi^*}(s')]$$

- Note that

$$\langle \theta_h^*, \phi(s_h^n, a_h^n) \rangle \approx r_h^n + V_{h+1}^{\pi^*}(s_{h+1}^n)$$

Bellman optimality equations

$$V_h^{\pi^*}(s) = \max_{a \in A} Q_h^{\pi^*}(s, a)$$

$$Q_h^{\pi^*}(s, a) = r_h(s, a) + \mathbb{E}_{s' \sim \mathbb{P}_h(\cdot | s, a)} [V_{h+1}^{\pi^*}(s')]$$

Therefore, one may estimate $\theta_h^*, Q_h^{\pi^*}, V_h^{\pi^*}$ as

$$\hat{V}_H(s) = 0$$

$$\hat{\theta}_h = \underset{\theta}{\operatorname{argmin}} \left\{ \sum_{n=1}^N \left(\langle \phi(s_h^n, a_h^n), \theta \rangle - (r_h^n + \hat{V}_{h+1}(s_{h+1}^n)) \right)^2 + \lambda \|\theta\|^2 \right\}$$

$$= \left(\sum_{n=1}^N \phi(s_h^n, a_h^n) \phi(s_h^n, a_h^n)^\top + \lambda I \right)^{-1} \left(\sum_{n=1}^N \phi(s_h^n, a_h^n) (r_h^n + \hat{V}_{h+1}(s_{h+1}^n)) \right)$$

$$\hat{Q}_h(s, a) = \langle \hat{\theta}_h, \phi(s, a) \rangle \quad \Lambda_h$$

$$\hat{\pi}_h(s) = \operatorname{argmax}_{a \in A} \hat{Q}_h(s, a)$$

$$\hat{V}_h(s) = \max_{a \in A} \hat{Q}_h(s, a)$$

Least square value iteration (LSVI)

Algorithm 4 Least square value iteration

Input: Datasets $\mathcal{D}_0, \dots, \mathcal{D}_{H-1}$.

- 1: Set $\hat{V}_H(\cdot) \leftarrow 0$.
- 2: **for** step $h = H - 1, \dots, 0$ **do**
- 3: Set $\Lambda_h \leftarrow \sum_{n=1}^N \phi(s_h^n, a_h^n) \phi(s_h^n, a_h^n)^\top + \lambda I$.
- 4: Set $\hat{\theta}_h \leftarrow \Lambda_h^{-1} \left(\sum_{n=1}^N \phi(s_h^n, a_h^n) \left(r_h^n + \hat{V}_{h+1}(s_h^n) \right) \right)$.
- 5: Set $\hat{Q}_h(\cdot, \cdot) \leftarrow \langle \hat{\theta}_h, \phi(\cdot, \cdot) \rangle$.
- 6: Set $\hat{\pi}_h(\cdot | s) \leftarrow \arg \max_{p \in \Delta(\mathcal{A})} \langle \hat{Q}_h(s, \cdot), p(\cdot) \rangle_{\mathcal{A}}$ for all $s \in \mathcal{S}$.
- 7: Set $\hat{V}_h(s) \leftarrow \langle \hat{Q}_h(s, \cdot), \hat{\pi}_h(\cdot | s) \rangle_{\mathcal{A}}$ for all $s \in \mathcal{S}$.
- 8: **end for**

Output: $(\hat{\pi}_h)_{h=0}^{H-1}$.

$$\hat{V}_H(s) = 0$$

$$\begin{aligned} \hat{\theta}_h &= \underset{\theta}{\operatorname{argmin}} \left\{ \sum_{n=1}^N \left(\langle \phi(s_h^n, a_h^n), \theta \rangle - (r_h^n + \hat{V}_{h+1}(s_h^n)) \right)^2 + \lambda \|\theta\|^2 \right\} \\ &= \left(\sum_{n=1}^N \phi(s_h^n, a_h^n) \phi(s_h^n, a_h^n)^\top + \lambda I \right)^{-1} \left(\sum_{n=1}^N \phi(s_h^n, a_h^n) (r_h^n + \hat{V}_{h+1}(s_h^n)) \right) \end{aligned}$$

$$\hat{Q}_h(s, a) = \langle \hat{\theta}_h, \phi(s, a) \rangle \quad \Lambda_h$$

$$\hat{\pi}_h(s) = \underset{a \in A}{\operatorname{argmax}} \hat{Q}_h(s, a)$$

$$\hat{V}_h(s) = \max_{a \in A} \hat{Q}_h(s, a)$$

- In online setting, we may combine LSVI with UCB

Algorithm 1 LSVI-UCB

for episode $k = 1, 2, \dots, K$ **do**

1. (Value estimation) **for** step $h = H, H - 1, \dots, 1$ **do**

(a) (Gram matrix) $\Lambda_h^k \leftarrow \sum_{\tau \in [k-1]} \phi(x_h^\tau, a_h^\tau) \phi(x_h^\tau, a_h^\tau)^\top + I$

(b) (Least squares)

$$\begin{aligned} w_h^k &\leftarrow \arg \min_{w \in \mathbb{R}^d} \sum_{\tau \in [k-1]} [r_h(x_h^\tau, a_h^\tau) + V_{h+1}^k(x_{h+1}^\tau) - \langle w, \phi(x_h^\tau, a_h^\tau) \rangle]^2 + \|w\|^2 \\ &= (\Lambda_h^k)^{-1} \sum_{\tau \in [k-1]} \phi(x_h^\tau, a_h^\tau) \cdot [r_h(x_h^\tau, a_h^\tau) + V_{h+1}^k(x_{h+1}^\tau)]. \end{aligned}$$

(c) (Q estimate with UCB) $Q_h^k(\cdot, \cdot) = \langle w_h^k, \phi(\cdot, \cdot) \rangle + \beta \sqrt{\phi(\cdot, \cdot)^\top (\Lambda_h^k)^{-1} \phi(\cdot, \cdot)}$

(d) (From Q to value function) $V_h^k(\cdot) = \max_a Q_h^k(\cdot, a)$.

2. Receive initial state x_1^k

3. (Policy execution) **for** step $h = 1, 2, \dots, H$ **do**

Take action $a_h^k \leftarrow \arg \max_a Q_h^k(x_h^k, a)$; observe reward $r_h^k = r_h(x_h^k, a_h^k)$ and next state x_{h+1}^k .

Take home messages

- Self-taught reasoning in LLM may be (partly) explained by a Markov chain.
- With properly designed step size, GD for smooth PL loss functions may still converge to optimal under adversarial setting.
- Optimism in the face of uncertainty (OFU) principle
- Regret bound analysis of UCB in linear bandit problem.
- LSVI/LSVI-UCB for reinforcement learning.